

UNIVERSITÀ
DI PARMA

Progetto sulla percolazione

AUTORE
Di Blasi Giuseppe

Modellazione e Simulazioni Numeriche

Indice

1 Introduzione **1**

2 Algoritmo Cluster Finding visto a lezione **2**
2.1 Inizializzazione . . . 2
2.2 Loop ricerca cluster . . . 3
2.3 Pro e contro dell'algoritmo di cluster finding visto a lezione . . . 4

3 Algoritmo Hoshen–Kopelman **5**
3.1 Struttura Union-Find . . . 5
3.2 Implementazione algoritmo HK . . . 5
3.2.1 Fase di inizializzazione . . . 5
3.2.2 Ciclo di etichettatura dei cluster . . . 7
3.2.3 Funzioni Union e Find . . . 8
3.2.4 Correzione delle etichette . . . 10
3.2.5 Conteggio dei cluster . . . 10
3.2.6 Riepilogo . . . 10

4 Confronto prestazioni tra i due algoritmi **11**
4.1 Premessa . . . 11
4.2 Preparazione del benchmark . . . 11
4.2.1 Verifica della correttezza dell'implementazione dell'algoritmo HK . . . 11
4.3 Benchmark con p_{col} fissa e taglia variabile . . . 13
4.3.1 Risultato benchmark . . . 14
4.3.2 Stesso benchmark, ma con reticoli più grandi . . . 14
4.4 Benchmark con taglia fissa e p_{col} variabile . . . 16

5 Determinazione probabilità di percolazione **18**
5.1 Struttura dello script . . . 18
5.2 Determinazione probabilità di percolazione . . . 18
5.2.1 Percolazione Left-Right e Top-Bottom . . . 21
5.3 Determinazione delle quattro osservabili . . . 21
5.3.1 Osservabile P1 . . . 24
5.3.2 Osservabile P2 . . . 26
5.3.3 Osservabile P3 . . . 28
5.3.4 Osservabile RACS . . . 29

6 Conclusioni **31**

7 Appendice **33**

1 Introduzione

Questa relazione porta avanti il progetto sulla teoria della percolazione che è stato affrontato durante le lezioni del corso.

Le simulazioni sulla percolazione permettono di modellare vari fenomeni reali come:

- Passaggio dei fluidi all'interno di materiali porosi
- Studio sulla frammentazione di popolazioni animali
- Prevedere la frammentazione dei gusci (capsidi) di alcuni virus
- Di recente è stata utilizzata anche per studiare l'andamento del traffico all'interno delle città

La simulazione tramite percolazione prevede l'utilizzo di un reticolo 2D o 3D, di grandezza $L \times L$ o $L \times L \times L$ dove i nodi rappresentano dei siti che possono essere colorati secondo una probabilità p_{col}.

Possiamo vedere inizialmente il reticolo come una rete di siti non colorati, procedendo con il processo di colorazione la rete si frammenta in *cluster* di dimensioni variabili.

Un aspetto fondamentale della percolazione è l'esistenza di una probabilità critica p_c, detta soglia di percolazione, oltre la quale compare un cluster che attraversa l'intero reticolo.

Dato che lo studio analitico di questi fenomeni risulta altamente complesso, soprattutto per reticoli con dimensioni elevate, ci si affida a simulazioni numeriche per semplificare lo studio del fenomeno della percolazione.

Nelle simulazioni svolte su MATLAB a lezione abbiamo scelto di utilizzare reticoli di grandezza *L+2* per aggiungere una cornice di padding per permetterci di semplificare il controllo delle condizioni a contorno. La cornice di padding è stata poi rimossa una volta terminata la colorazione dei siti del reticolo. Più nel dettaglio abbiamo creato una struct *res* che contiene tutti i dati relativi alla percolazione:

- *p*, probabilità di colorare un sito
- *matrice*, matrice dei siti colorati
- *label*, matrice delle etichette dei cluster trovati
- *percolazioneTB*, valore di esistenza della percolazione Top-Bottom
- *percolazioneLR*, valore di esistenza della percolazione Left-Right

L'obiettivo della relazione è implementare l'algoritmo Hoshen–Kopelman per il cluster finding, confrontarlo con l'algoritmo di cluster finding utilizzato a lezione, confrontare la correttezza e i tempi d'esecuzione dei due algoritmi in determinate condizioni. Inoltre la relazione prevede anche il calcolo della soglia di percolazione con i relativi errori.

Nota: A volte potrebbe essere usato il termine **colorare** o **etichettare**, sono sinonimi intercambiabili.

2 Algoritmo Cluster Finding visto a lezione

Durante il corso abbiamo visto l'algoritmo per la colorazione dei siti. Questo algoritmo sfrutta una pila per tenere traccia del reticolo attuale e guarda i 4 vicini del sito che stiamo attualmente visitando.

Vediamo la parte di inizializzazione dell'algoritmo:

2.1 Inizializzazione

Listing 1: Inizializzazione Algoritmo

```
res.matrice = zeros(L + 2);
aux = rand(L) < p;
res.matrice(2:end-1, 2:end-1) = aux;

res.percolazioneTB = 0;
res.percolazioneLR = 0;

res.p = p;

res.label = zeros(L + 2);

res.clusters = [];

labelC = 1;

valid = find(res.matrice);
```

Nella parte di inizializzazione viene creata la matrice contenente i siti tramite la funzione zeros(L + 2), questo ci darà una matrice grande *(L + 2) x (L + 2)* con il bordo di padding come detto durante l'introduzione. Stesso procedimento per la matrice delle label.

Inizializziamo la matrice di zeri, tralasciando la cornice di padding, con dei valori casuali tra 1 e 0, che rappresentano i siti colorati: **1** per un sito colorato e **0** per un sito non colorato.

Inoltre salviamo dentro la variabile **valid** i siti che sono ad 1, così evitiamo di dover scorrere l'intero reticolo per trovare i siti da etichettare.

Guardiamo adesso il loop per la ricerca dei cluster:

2.2 Loop ricerca cluster

Listing 2: Loop ricerca cluster

```
    for iter = 1:length(valid)

    ii = valid(iter);
    if (res.label(ii) == 0)
        % nuovo cluster, esploro i vicini

        pila = ii;
        res.label(ii) = labelC;

        j = 1;
        while (j <= length(pila))
            elemento = pila(j);

            % aggiungo i vicini alla pila
            if (res.matrice(elemento - 1) && res.label(elemento - 1) == 0)
                pila(end + 1) = elemento - 1;
                res.label(elemento - 1) = labelC;
            end

            if (res.matrice(elemento + 1) && res.label(elemento + 1) == 0)
                pila(end + 1) = elemento + 1;
                res.label(elemento + 1) = labelC;
            end

            if (res.matrice(elemento - L - 2) && res.label(elemento - L - 2)
               == 0)
                pila(end + 1) = elemento - L - 2;
                res.label(elemento - L - 2) = labelC;
            end

            if (res.matrice(elemento + L + 2) && res.label(elemento + L + 2)
               == 0)
                pila(end + 1) = elemento + L + 2;
                res.label(elemento + L + 2) = labelC;
            end
            j = j + 1;
        end
        res.clusters(end+1) = length(pila);
        labelC = labelC + 1;
    end
end
```

Il cliclo for itera su tutti i siti validi; per ogni sito valido devo controllare se:

- Se il sito attuale non fa parte di un cluster, allora deve far parte dell'attuale cluster che sto cercando, viene aggiunto alla pila e si controlla se ha dei nodi vicini che possono far parte del cluster
- All'interno del while loop viene controllato se i vicini del nodo appena aggiunto sono siti che devono essere etichettati, ma non fanno parte di un cluster. In caso positivo vengono aggiunti all'attuale cluster che sti sta etichettando.
- Infine viene aggiunto all'interno di **res.clusters** la dimensione del cluster che è stato trovato. Viene aggiornato il contatore dell'attuale etichetta e il ciclo si ripete

2.3 Pro e contro dell'algoritmo di cluster finding visto a lezione

L'algoritmo visto a lezione risulta intuitivo da implementare, esplora un sito, e se necessario etichetta eventuali vicini che appartengono allo stesso cluster. Presenta una coplessità computazionale $O(N)$, dove *N* è il numero dei nodi all'interno del reticolo; in questa versione tramite **valid** riusciamo ad evitare di dover scansionare l´intera griglia, di conseguenza possiamo considerare *N* come il numero di siti validi. Una problematica che si può riscontrare è l'utilizzo della pila per tenere traccia del cluster. Per reticoli di dimensioni contenute non risulta un problema, ma si dovesse lavorare con reticoli di dimensioni elevate bisogna tenere in considerazione l'utilizzo aggiuntivo della memoria causato dall'utilizzo della pila.

3 Algoritmo Hoshen–Kopelman

Per questa relazione è richiesto di confrontare le prestazioni tra l´algoritmo di cluster finding visto a lezione e l'algoritmo Hoshen–Kopelman. L'algoritmo di Hoshen-Kopelman scansiona il reticolo per intero una sola volta. Dal punto di vista teorico l'algoritmo Hoshen–Kopelman ha una complessità computazionale
di $O(N \times \alpha(N))$, dove:

- N, numero di nodi sul reticolo
- α, funzione inversa di Ackerman, ha una crescita molto lenta che permette di definire α come quasi costante

3.1 Struttura Union-Find

All'apparenza, data la complessità computazionale, i due algoritmi possono sembrare equivalenti sulla carta, però l'algoritmo HK permette di evitare l'utilizzo di uno stack risolvendo il problema legato ad un maggior utilizzo di memoria con cluster di dimensioni elevate. Al posto della pila sfrutta una struttura **union-find** per risolvere eventuali conflitti tra i cluster. Questa struttura **union-find** permette di unire 2 cluster con etichette diverse che dovrebbero formare un unico cluster. Grazie alla **path-compression** possiamo rendere il tempo di unione dei due cluster quasi costante.

3.2 Implementazione algoritmo HK

3.2.1 Fase di inizializzazione

Possiamo notare che la fase di inizializzazione è leggermente diversa rispetto a quella dell'algoritmo visto a lezione.

Listing 3: Inizializzazione algoritmo HK

```
res.matrice = zeros(L + 2);
aux = rand(L) < p;

res.matrice(2:end-1, 2:end-1) = aux;

res.percolazioneTB = 0;
res.percolazioneLR = 0;

res.p = p;

res.label = zeros(L + 2);

res.clusters = [];

labelC = 1;

radici_cluster = 1:(L*L);
```

L'unica cosa diversa che notiamo è la comparsa di **radici_cluster**, un vettore riga con una dimensione $1 \times L^2$, in questo caso non abbiamo bisogno del padding visto che all'interno di questo vettore metteremo le radici dei cluster e i suoi elementi che lo compongono. Qui non viene usata la variabile **valid** dato che l'algoritmo HK scorre l'intero reticolo una sola volta, questa è una scelta implementativa per cercare di confrontare l'algoritmo visto a lezione con HK puro. Guardiamo adesso come è cambiato il loop per la ricerca dei siti da etichettare.

3.2.2 Ciclo di etichettatura dei cluster

Listing 4: Ciclo per etichettatura dei cluster

```
    for rig = 2:L+1
        for col = 2:L+1

            if(res.matrice(rig, col) == 0)
                continue;
            end

            vicino_sup = res.label(rig - 1, col);
            vicino_sx = res.label(rig, col - 1);

            % Trovato nuovo cluster
            if vicino_sup == 0 && vicino_sx == 0
                res.label(rig, col) = labelC;
                radici_cluster(labelC) = labelC;
                labelC = labelC + 1;

            % Estendo il cluster in verticale
            elseif vicino_sup > 0 && vicino_sx == 0
                res.label(rig, col) = vicino_sup;

            % Estendo il cluster in orizzontale
            elseif vicino_sup == 0 && vicino_sx > 0
                res.label(rig, col) = vicino_sx;

            % Vicini con label diverse, controllo se devo unire i cluster
            else
                res.label(rig, col) = vicino_sx;

            % Devo unire i due cluster
            if vicino_sx ~= vicino_sup
                unisci_cluster(vicino_sup, vicino_sx);
            end
        end
    end
end
```

Possiamo notare che la maggior parte della logica è rimasta quasi identica, tranne per il controllo dei vicini, infatti l'algoritmo HK controlla solo il vicino superiore e il vicino di sinsitra per espandere un cluster.

Anche l'etichettatura del cluster è diversa rispetto all'algoritmo visto a lezione, infatti qui se incontriamo un sito da etichettare libero, lo etichettiamo con il valore di **labelC** e poi controlliamo il suo vicino superiore e il suo vicino sinistro. La radice del nuovo cluster viene salvata in **radici_cluster(labelC)**

Un caso interessante dell'algoritmo HK è l'unione di due cluster, infatti qui entrano in gioco le funzioni *find(x)* e *union(x, y)* che ci permettono di unire questi due cluster in un tempo quasi costante, dovuta alla path-compression implicita che ha complessità computazionale $O(\alpha(N))$.

Adesso guardiamo le funzioni *find(x)* e *union(x, y)* nel caso in cui si debbano unire due cluster diversi.

3.2.3 Funzioni Union e Find

Come già detto precedentemente, grazie a queste due funzioni possiamo unire due cluster con un tempo quasi costante. Queste due funzioni sono state implementate come funzioni locali all'interno del file *CercaClusterHK.m*, la scelta di usarle come funzioni locali è che ovviamente non ha senso esporle all'esterno visto che sono funzioni di supporto per l'algoritmo.
Iniziamo guardando l'implementazione della funzione *union(x)* che nel codice Matlab prende il nome di *unisci_cluster(x, y)*.

Listing 5: Funzione union algoritmo HK

```
1    function unisci_cluster(x, y)
2    % x e y sono le etichette dei due cluster da unire
3        rx = findRadice(x);
4        ry = findRadice(y);
5
6         if rx < ry
7            radici_cluster(ry) = rx;
8        elseif ry < rx
9            radici_cluster(rx) = ry;
10       end
11
12   % L'algoritmo prevede solo la riga qui sotto
13   % La parte superiore a questo commento e' stata aggiunta per unire i
14   % cluster in modo che la radice piu' grande, quella aggiunta
        successivamente,
15   % venga aggiunta alla radice
16   % con valore minore, aggiunta per prima
17   % --> radici_cluster(findRadice(x)) = findRadice(y);
18   end
```

La funzione *unisci_cluster(x, y)* prende in input due parametri, **x** e **y**, che sono rispettivamente i valori di **vicino_sup** e **vicino_sx**. C'è da tenere in considerazione che l'algoritmo originale prevede solamente la riga n.17 nel codice riportato appena sopra. La modifica effettuata serve solamente per migliorare l'estetica delle label una volta uniti i cluster. Nel concreto questa scelta è dovuta al fatto che veniva sempre usato il numero di label più alto per unire il cluster, con questa modifica verrà scelto sempre il numero pìu basso in modo da "rispettare" il cluster creato per primo. la funzione *unisci_cluster(x, y)* si appoggia all'implementazione della funzione *find(x)*, ovvero *findRadice(x)* per trovare gli elementi da unire al cluster precedente.

Di seguito viene riportato il codice per la funzione *findRadice(x)*.

Listing 6: Funzione find algoritmo HK

```
1   function y = findRadice(x)
2       % x, label del cluster
3       y = x;
4
5       while radici_cluster(y) ~= y
6           y = radici_cluster(y);
7
8       end
9
10      % Secondo while: path compression
11      while radici_cluster(x) ~= x
12          z = radici_cluster(x);
13          radici_cluster(x) = y;
14          x = z;
15      end
16
17  end
```

La funzione *findRadice(x)* prende in input la label del cluster x per trovarne la radice. Con il primo while risaliamo la catena del cluster fino a trovare la radice del cluster che ci interessa, ovvero quando y è uguale a se stesso.

Una volta raggiunta la radice del cluster possiamo eseguire la path compression, questo permette di ottimizzare la struttura ad albero implicita per velocizzare la ricerca delle prossime radici.

Nel secondo while colleghiamo i nodi del cluster che parte da x con radice y. Così facendo ottimizziamo il percorso per il cluster con radice **y**.

3.2.4 Correzione delle etichette

La struttura union-find non lavora direttamente su **res.label** ma sfrutta **radici_cluster** per tenere traccia di quali cluster unire. Detto questo siamo costretti a sistemare le etichette dei cluster dopo la prima scansione. Vediamo il cliclo for per questa operazione

Listing 7: Ciclo correzione label

```
    for i = 2:L+1
    for j = 2:L+1
        if res.label(i,j) > 0
            res.label(i,j) = findRadice(res.label(i,j));
        end
    end
end
```

3.2.5 Conteggio dei cluster

Un'altra differenza significativa con l'algoritmo di cluster finding visto a lezione e che con Hoshen-Kopelman dobbiamo contare i cluster trovati una volta terminato il processo di etichettatura, dovuto al fatto che non utilizziamo più la pila come supporto.

Viene riportato di seguito il codice per il conteggio dei cluster, sempre all'interno del file *Cerca-ClusterHK.m*.

Listing 8: Conteggio cluster in HK

```
    % Conto i cluster, questo tocca farlo dopo aver riordinato le etichette

    lblUniq = unique(res.label(res.label > 0));
    res.clusters = histcounts(res.label(:), [lblUniq; max(lblUniq)+1]);
```

3.2.6 Riepilogo

Dal punto di vista teorico l'algoritmo Hoshen-Kopelman ci garantisce prestazioni superiori con reticoli di dimensioni elevate, grazie anche alla path compression, velocizza la ricerca delle radici dei cluster. L'implementazione dell'algoritmo visto a lezione risulta ottimale per reticoli di dimensioni contenute, anche perché più intuitiva da implementare. Il prossimo capitolo della relazione affronterà le differenze prestazionali tra i due algoritmi presentati.

4 Confronto prestazioni tra i due algoritmi

4.1 Premessa

Questo capitolo affronta il confronto prestazionale tra l'algoritmo Hoshen-Kopelman e l'algoritmo di cluster finding implementato a lezione. Dal punto di visto teorico, come già anticipato precedentemente, ci aspettiamo delle prestazioni migliori da parte dall'algoritmo HK. Il benchmark è suddiviso in due fasi:

- Studio dell'andamento dei tempi con probabilità di colorazione del sito fissata e con grandezza del reticolo variabile.
- Studio dell'andamento dei tempi con grandezza del reticolo fissata e probabilità di colorazione dei siti variabile.

Ovviamente i reticoli dovranno essere uguali per entrambi gli algortmi per rispettare la correttezza del benchmark.

La probabilità di colorazione verrà chiamata p_{col}

4.2 Preparazione del benchmark

Le due funzioni *CercaCluster(L, p)* e *CercaClusterHK(L, p)* sono state modificate per poter accettare reticoli creati dall'esterno. Le due funzioni diventano quindi: *CercaClusterBenchmark(L, reticolo, p)* e *CercaClusterHKBenchmark(L, reticolo, p)*. Questo permette di poter ricevere reticoli creati fuori dalle funzioni e confrontare i tempi dei due algoritmi per lo stesso reticolo. Questo è dovuto al fatto che anche fissando il seed ad un numero n con la funzione *rng(*n*)* la chiamata *rand(*L*)* all'interno delle funzioni genera reticoli diversi compromettendo la correttezza del benchmark. Per una maggior correttezza del benchmark è stato inoltre fissato il seed per la generazione casuale a *rng(1)* per tutte le consegne successive, questo permette di verificare il comportamento dei due algoritmi con gli stessi reticoli, e di replicare i risultati.

4.2.1 Verifica della correttezza dell'implementazione dell'algoritmo HK

Prima di procedere con la discussione dei due benchmark è importante parlare della correttezza dell'implementazione dell'algoritmo HK.

Grazie alle due nuove funzioni è stato possibile passare lo stesso reticolo per verificare la correttezza dell'algoritmo HK.

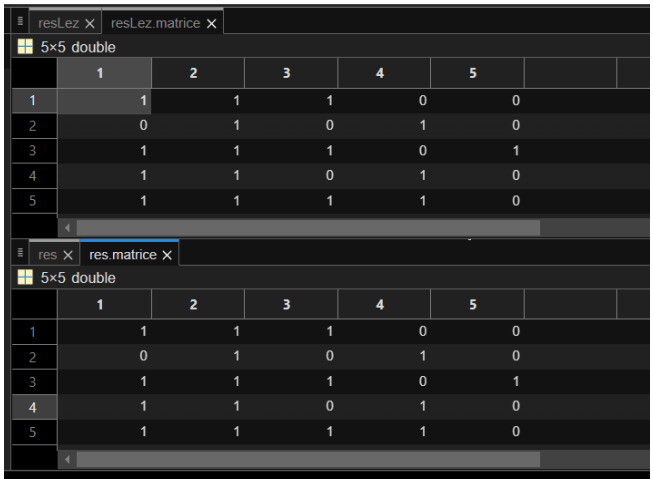


Figura 1: Matrice con i siti da colorare

Per questioni di praticità è stato scelto $L = 5$ in modo da poter mostrare i risultati in modo comprensibile all'interno di questa relazione. Possiamo vedere dall'immagine riportata qui sopra che le funzioni hanno lavorato sulla stessa matrice, di conseguenza dovremmo aspettarci che abbiamo colorato i siti in egual modo.

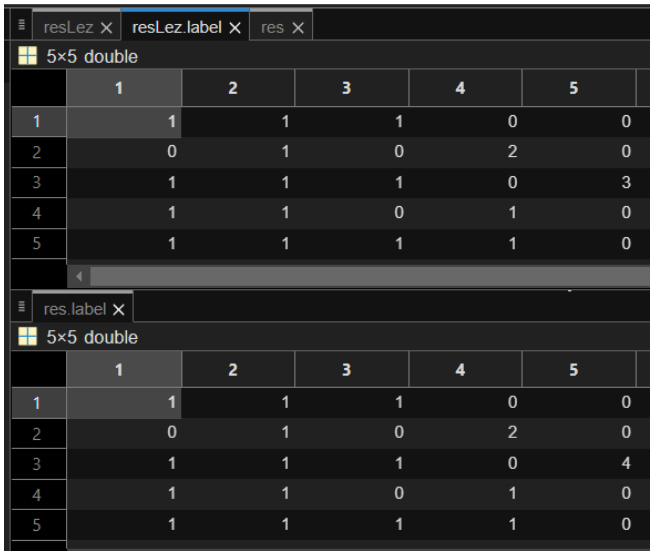


Figura 2: Matrice delle etichette per i cluster

Dalla figura qui sopra notiamo che i due algoritmi hanno trovato gli stessi cluster, ma le etichette sono leggermente diverse. Questo non è un errore di correttezza nel senso stretto, ma semplicemente un errore di stile che non va ad intaccare la correttezza dell'algoritmo HK.

In conclusione possiamo confermare che l'implementazione dell'algoritmo HK risulta corretta nonostante una discrepanza nello stile di etichettatura dei cluster dovuta alla diversa logica di etichettatura dell'algoritmo HK.

4.3 Benchmark con p_{col} fissa e taglia variabile

Partiamo con spiegare i dati che servono per questo benchmark:

Nome variabile	Descrizione	Valore
L	Dimensione del reticolo di partenza.	30:50:400
R	Numero di iterazioni sullo stesso reticolo.	2000
p	Probabilità di colorazione di un sito.	0.55
tempiHK	matrice contenete i tempi per l'algoritmo HK.	zeros(length(L), R)
tempiLez	matrice contenente i tempi per l'algoritmo visto a lezione.	zeros(length(L), R)

Durante il benchmark verranno calcolati i seguenti dati:

Nome variabile	Descrizione
tempoMedioHK	Contiene il tempo medio d'esecuzione dell'algoritmo HK.
tempoMedioLez	Contiene il tempo medio d'esecuzione dell'algoritmo vistoa lezione.
erroreHK	Contiene l'errore standard medio dell'algoritmo HK.
tempiLez	Contiene l'errore standard medio dell'algoritmo visto a lezione

4.3.1 Risultato benchmark

Riportiamo qui sotto il grafico che ci mostra i risultati del benchmark con i dati mostrati precedentemente.

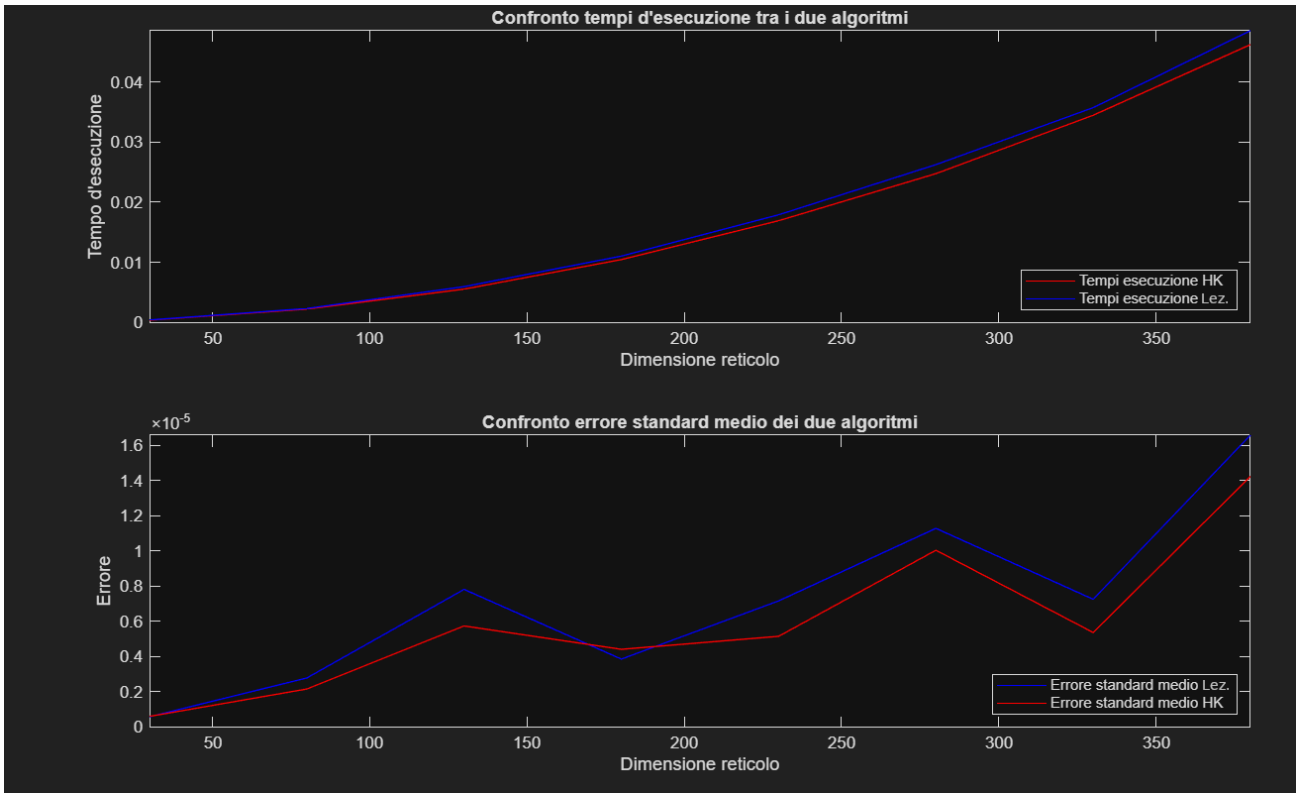


Figura 3: Confronto HK vs Lez.

Nel grafico dei tempi notiamo che l'algoritmo HK tende ad essere più veloce con la crescita del reticolo. Questo è un andamento che ci aspettavamo, però la dimensione dei reticoli è ancora piuttosto contenuta per dare un giudizio definitivo.

Nel grafico dell'errore standard medio notiamo che l'algoritmo HK tende ad avere un errore più basso rispetto a quello dell'algoritmo visto a lezione.

4.3.2 Stesso benchmark, ma con reticoli più grandi

Per confrontare meglio l'andamento dei tempi con dei casi più "signficativi" è stata modificata la variabile **L** per ottenere dei reticoli di dimensioni più elevate. Per questo benchmark **L** avrà i seguenti valori **L = [15000, 20000, 25000];**.
Prima di procedere, l'algoritmo HK è stato implementato in modo piuttosto diretto, seguendo l'algritmo originale. Ovviamente con dei reticoli così grandi potremmo sfruttare **valid** come fa l'algoritmo

UNIVERSITÀ DI PARMA

visto durante le lezioni del corso. Così lavoreremo solo sui siti da colorare evitando iterazioni inutili. Per questo benchmark verrà messo a confronto la versione originale HK, con l'algoritmo visto a lezione e con la versione ottimizzata di HK, in modo da confrontare tempi e errori medi standard dei 3 algoritmi.

L'unica cosa da tenere in considerazione è che l'algoritmo HK originale lavora in row-major order, quindi scannerizziamo riga per riga. La versione ottimizzata che sfrutta valid lavora in column-major order, questo vuol dire che avremo label leggermente diverse, ma non intacca la correttezza dell'algoritmo.

Guardiamo il grafico dei tempi nella pagina che segue

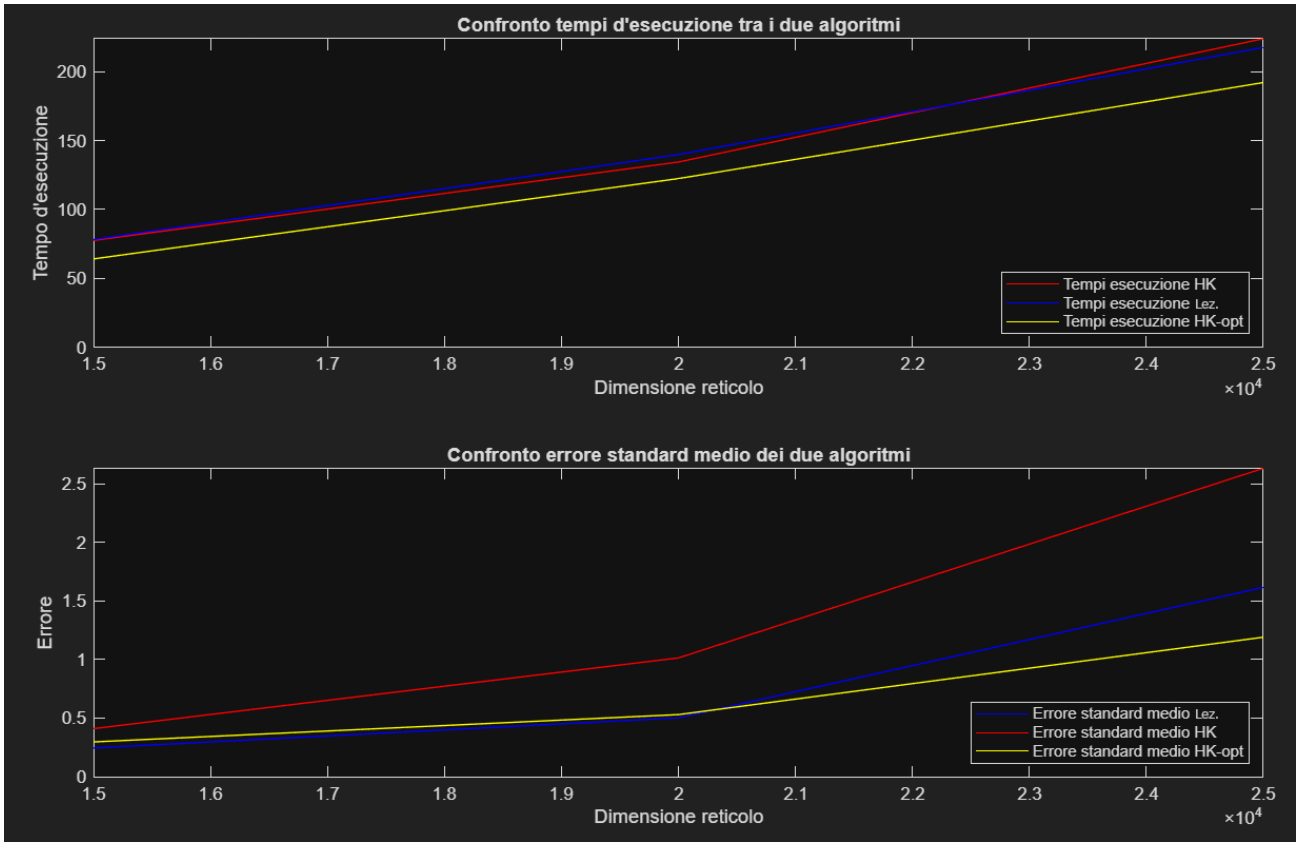


Figura 4: Grafico con algoritmo HK ottimizzato

Possiamo notare che con l'ottimizzazione fatta sull'algoritmo HK di partenza, ovvero facendolo lavorare direttamente sui siti da colorare invece di scannerizzare l'intera griglia ci ha permesso di abbassare drasticamente i tempi rispetto all'implementazione originale. Stessa cosa vale per l'errore standard medio.

4.4 Benchmark con taglia fissa e p_{col} variabile

In questo benchmark i valori visti precedentemente sono leggermente diversi.

Nome variabile	Descrizione	Valore
L	Dimensione del reticolo di partenza.	200
R	Numero di iterazioni sullo stesso reticolo.	2000
p	Probabilità di colorazione di un sito.	0.25:0.05:0.95
tempiHK	matrice contenete i tempi per l'algoritmo HK.	zeros(length(p), R)
tempiLez	matrice contenente i tempi per l'algoritmo visto a lezione.	zeros(length(p), R)
tempiHKOpt	matrice contenente i tempi per l'algoritmo HK ottimizzato.	zeros(length(p), R).

Durante il benchmark verranno calcolati i seguenti dati:

Nome variabile	Descrizione
tempoMedioHK	Contiene il tempo medio d'esecuzione dell'algoritmo HK.
tempoMedioLez	Contiene il tempo medio d'esecuzione dell'algoritmo visto a lezione.
tempoMedioHKopt	Contiene il tempo medio d'esecuzione dell'algoritmo HK ottimizzato.
erroreHK	Contiene l'errore standard medio dell'algoritmo HK.
erroreLez	Contiene l'errore standard medio dell'algoritmo visto a lezione.
erroreHKOpt	Contiene l'errore standard medio dell'algoritmo HK ottimizzato.

Da questo benchmark ci aspettiamo di vedere degli andamenti interessanti quando **p** è attorno alla soglia di percolazione, circa 0.6.

Diamo un'occhiata al grafico con **p**, probabilità di colorazione fissa su un reticolo di dimensione **L** fissa.

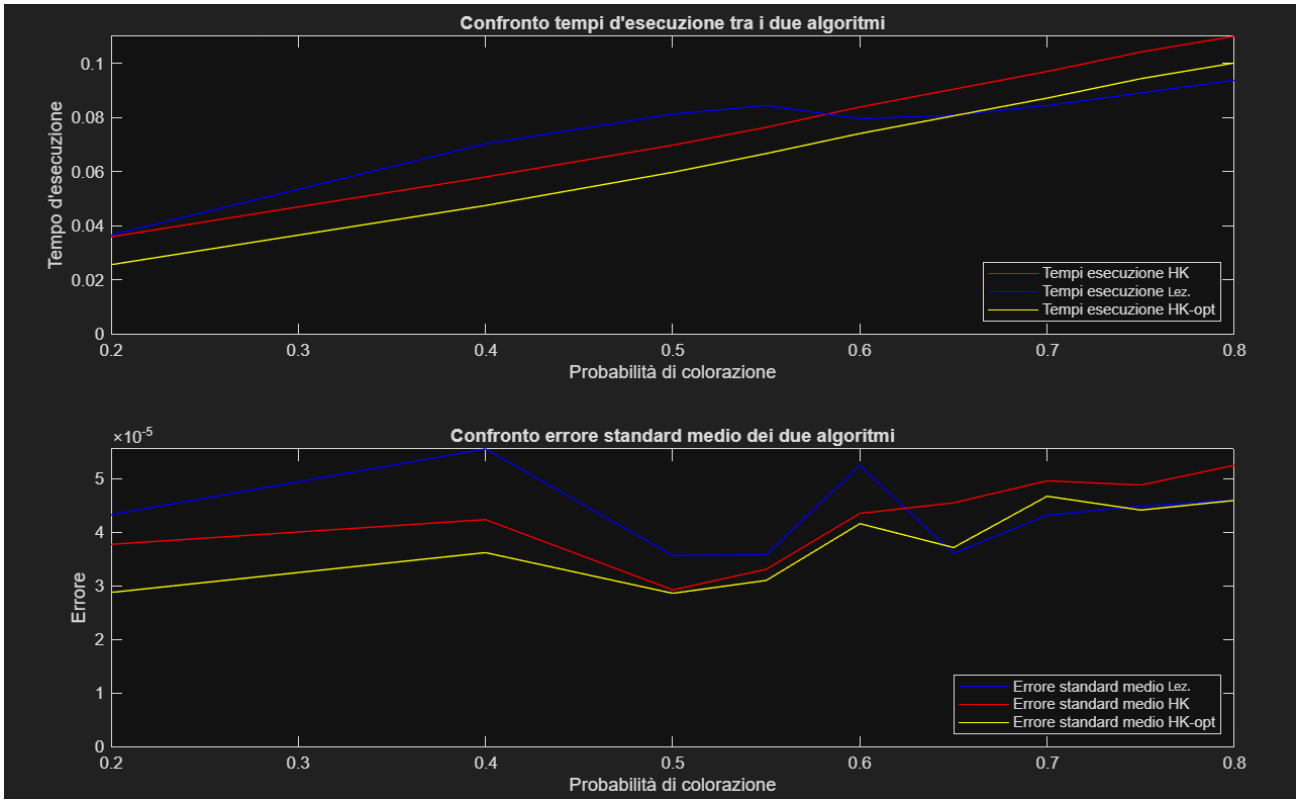


Figura 5: Grafico prestazioni con probabilità variabile

Dal grafico possiamo notare come l'andamento temporale dei due algoritmi HK non risentono del cambio della probabilità di colorazione e mantengono lo stesso andamento temporale. Invece, l'algoritmo visto a lezione, quando viene superata la soglia critica, diventa più efficiente risultanto anche leggermente più veloce della versione HK ottimizzata.

UNIVERSITÀ DI PARMA

5 Determinazione probabilità di percolazione

In questo ultima sezione lavoreremo sulla consegna che riguarda la determinazione della soglia di percolazione con diverse probabilità di colorazione dei siti e con diverse taglie di reticoli. Inoltre verra calcolata anche la probabilità di percolazione sinistra-destra e superiore-inferiore. Dimostreremo come cambi la probabilità di percolazione al cambiare della grandezza del reticolo: ci aspettiamo che reticoli più grandi inizino a percolare più lentamente rispetto a reticoli più piccoli. Verranno inoltre calcolate anche quattro "osservabili" che ci permettono di dire qualcosa in più per quanto riguarda i reticoli percolanti.

5.1 Struttura dello script

Per determinare tutti i dati di cui abbiamo discusso precedentemente lavoreremo su diverse taglie per il reticolo e valori diversi della probablità di colorazione di un sito; questo per stabilire l'andamento delle quattro osservabili e della soglia di percolazione al cambiare dei dati. É stato usato lo script *DatiPERC.m* per calcolare tutti questi dati. Come algoritmo per la risoluzione verrà utilizzata la forma ottimizzata dell'algoritmo HK.

Per lo script *DatiPERC.m* sono stati scelti i seguenti dati:

Nome variabile	Descrizione	Valore
LL	Dimensione dei reticoli	[50, 100, 150]
R	Numero di iterazioni sullo stesso reticolo.	1000
p	Probabilità di colorazione di un sito.	... 0.3:0.005:0.7 ...

Per **p** sono stati scelti molti valori "interessanti" vicino alla soglia di percolazione per guardare l'andamento in quella zona della curva.

5.2 Determinazione probabilità di percolazione

Per determinare la probabilità di percolazione al variare di **p** è stata calcolata la presenza di percolazione per ogni valore di **p** per ogni iterazione **R** sul reticolo preso in considerazione. Nella pagina che segue viene riportato il codice MATLAB utilizzato per il calcolo della probabilità di percolazione.

Listing 9: Calcolo probabilità percolazione

```
    %...
    sommaProbPerc = 0;
    for ...
    %...

        perc = res.percolazioneLR || res.percolazioneTB;
        sommaProbPerc = sommaProbPerc + perc;

    %...
    end
    %...

    prob(p_idx) = sommaProbPerc / R;
    erroreProb(p_idx, L_idx) = sqrt(prob(p_idx) * (1 - prob(p_idx)) / R);

    %...
```

Possiamo vedere che la probabilità di percolazione è semplicemente il numero di volte in cui avviene la percolazione su **R** iterazioni. Inoltre è stato calcolato anche l'errore standard medio per la probabilità di percolazione, sfruttando la formula dell'errore standard medio:

$$Var(X) = p(1-p)$$

$$ESM = \sqrt{\frac{Var(X)}{R}}$$

Si può vedere il fenomeno della percolazione come un processo di Bernoulli e quindi si può sfruttare la definizione di varianza per una distribuzione bernoulliana.

Nella pagina che segue viene riportato un grafico dell'andamento della probabilità di percolazione al variare della grandezza del reticolo.

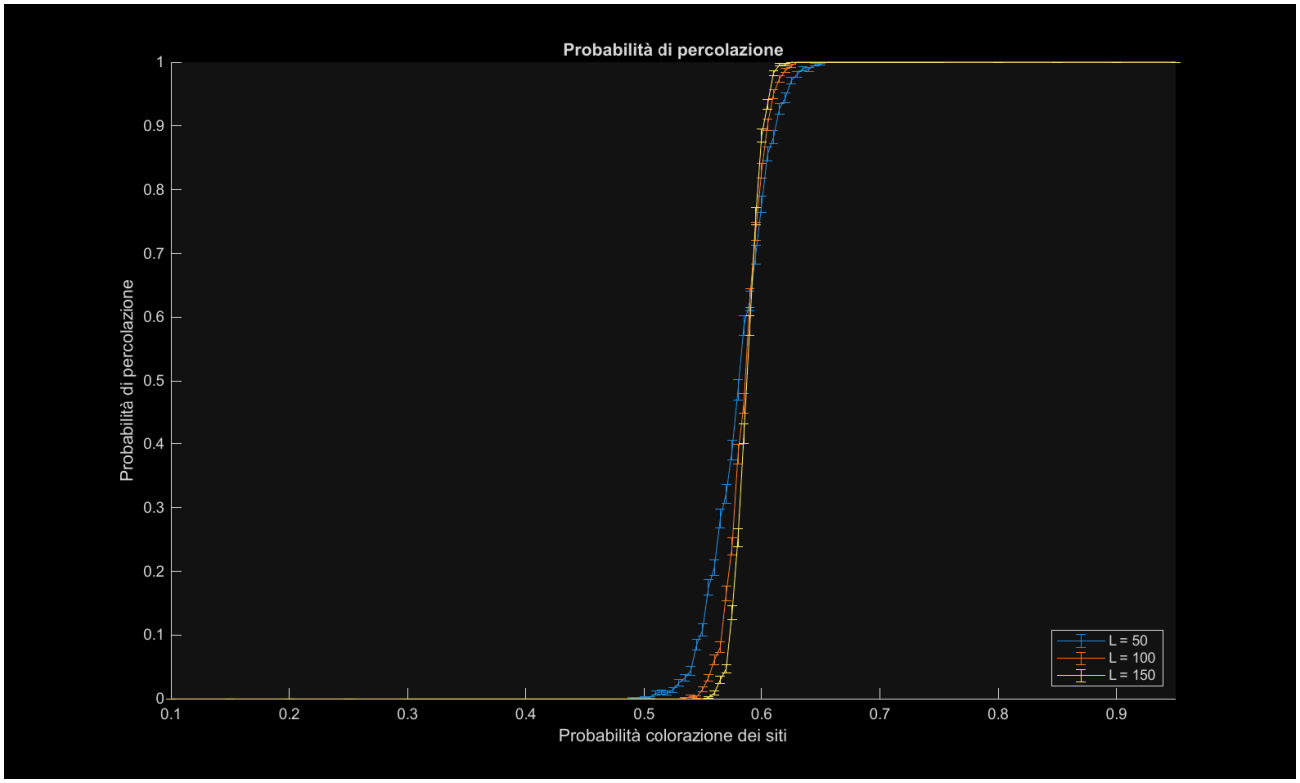


Figura 6: Grafico probabilità di percolazione

Possiamo notare che le curve mostrano un comportamento dipendente dalla dimensione del reticolo. Per reticoli più grandi, la transizione verso lo stato percolante avviene in modo più netto e concentrato attorno a un valore specifico di probabilità di colorazione. Tutte e 3 le curve si incontrano circa a 0.6 come valore di probabilità di colorazione dei siti. Inoltre, al crescere della dimensione del reticolo, le curve diventano sempre più ripide.

5.2.1 Percolazione Left-Right e Top-Bottom

È stato studiato anche l'andamento della probabilità di percolazione Left-Right (LR) e Top-Bottom (TB). Viene riportato il codice MATLAB estrapolato dallo script per mostrare come è stata calcolata.

Listing 10: Calcolo probabilità percolazione

```
    %...
    sommaLR = 0;
    sommaTB = 0;
    for ...
        %...

        percLR = res.percolazioneLR;
        percTB = res.percolazioneTB;

        %...
    end
    %...

    probPercLR(p_idx) = sommaLR / R;
    probPercTB(p_idx) = sommaTB / R;
    %...
    erroreProbPercLR(p_idx, L_idx) = sqrt(probPercLR(p_idx) * (1 - probPercLR
        (p_idx)) / R);

    erroreProbPercTB(p_idx, L_idx) = sqrt(probPercTB(p_idx) * (1 - probPercTB
        (p_idx)) / R);

    %...
```

Anche in questo caso sono state sfruttate le formule indicate nella sezione che riguarda il calcolo della probabilità di percolazione. Per gli andamenti della percolazione LR e TB viene messa in appendice una tabella che riporta i risultati ottenuti(vedere Tab. 1).

5.3 Determinazione delle quattro osservabili

Come è gia stato detto nella sezione precedente sono state determinate le quattro osservabili per questo progetto. Viene riportato il codice MATLAB per vedere come sono state calcolate le quattro osservabili:

Listing 11: Determinazione quattro osservabili

```
    %...
    valoriP1 = zeros(1, length(R));
    valoriP2 = zeros(1, length(R));
    valoriP3 = zeros(1, length(R));
    valoriRACS = zeros(1, length(R));

    for iter = 1:R

    %...

    %Cluster con dimensione piu' alta
    maxCluster = max(res.clusters);

    % P1: rapporto tra cluster massimo e numero di siti
    valoriP1(iter) = maxCluster / LL(L_idx)^2;

    % P2: rapportro tra cluster massimo, e numero di siti per la
    % probabilita' di colorazine
    valoriP2(iter) = maxCluster / (p * (LL(L_idx)^2));

    % P3
    P3 = 0;
    %Posso riutilizzarla al denominatore del RACS
    mediaPesataP3 = 0;

    for c = 1:length(res.clusters)
        ns = length(find(res.clusters == res.clusters(c)));
        mediaPesataP3 = mediaPesataP3 + (res.clusters(c) * ns);
    end

    valoriP3(iter) = maxCluster / mediaPesataP3;
    %  Calcolo RACS
    idxMax = find(res.clusters == maxCluster, 1);
    cluster_noMax = res.clusters;
    cluster_noMax(idxMax) = [];

    numRACS = 0;

    for c = 1:length(cluster_noMax)
        ns = length(find(cluster_noMax == cluster_noMax(c)));
        numRACS = numRACS + (ns*(cluster_noMax(c))) * cluster_noMax(c);
    end

    valoriRACS(iter) = numRACS / mediaPesataP3;
    %...
    end
```

Listing 12: Determinazione errore delle osservabili

```
    %...

    % calcolo errore delle osservabili e valori medi
    mediaP1 = mean(valoriP1);
    mediaP2 = mean(valoriP2);
    mediaP3 = mean(valoriP3);
    mediaRACS = mean(valoriRACS);

    datiOsservabili(1, p_idx) = mediaP1;
    datiOsservabili(2, p_idx) = mediaP2;
    datiOsservabili(3, p_idx) = mediaP3;
    datiOsservabili(4, p_idx) = mediaRACS;

    erroreOsservabili(1, p_idx) = std(valoriP1) / sqrt(R);
    erroreOsservabili(2, p_idx) = std(valoriP2) / sqrt(R);
    erroreOsservabili(3, p_idx) = std(valoriP3) / sqrt(R);
    erroreOsservabili(4, p_idx) = std(valoriRACS) / sqrt(R);

    %...
    end
```

5.3.1 Osservabile P1

L'osservabile P1, definita come:

$$P1 = \frac{s_{max}}{L^2}$$

dove:

- s_{max}, dimensione del cluster più grande
- L^2, numero di siti all'interno del reticolo

Questa equazione rappresenta la probabilità di un qualunque sito di appartenere al cluster più grande all'interno del reticolo. Viene riportato un grafico che mostra l'andamento dell'osservabile P1 al variare della grandezza del reticolo e della probabilità di colorazione.

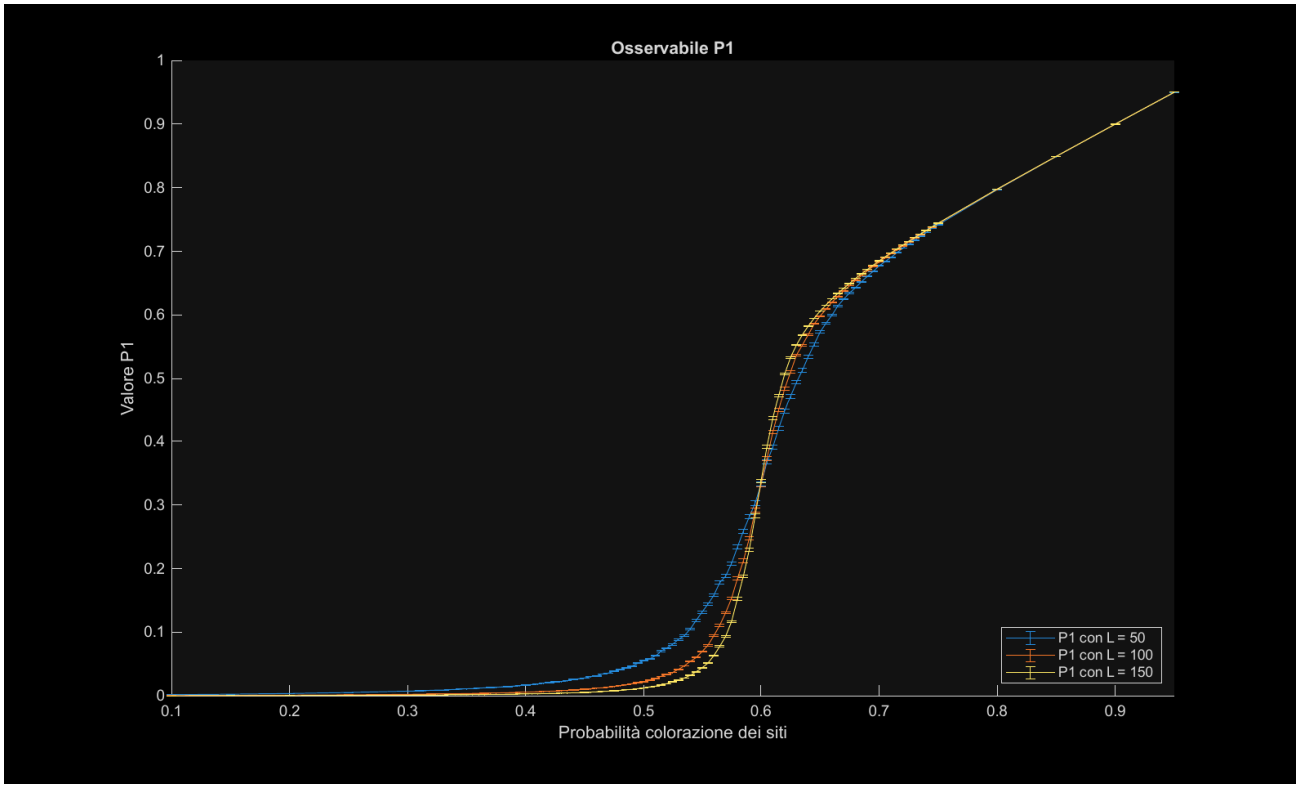


Figura 7: Osservabile P1

Come già detto l'osservabile P1 misura la probabilità di un qualunque sito di appartenere al cluster di dimensione più grande all'interno del reticolo. Si nota che con valori di L bassi si inizia ad avere una crescita per valori di colorazione più bassi. In tutti i casi, all'aumentare della probabilità di

UNIVERSITÀ DI PARMA

colorazione, P1 tende ad arrivare ad 1. Anche in questo caso tutte le curve si incontrano su un valore molto vicino a 0.6 per la probabilità di colorazione dei siti. Si nota anche che dopo la probabilità di colorazione pari a 0.7, tutte le curve tendono a sovrapporsi seguendo lo stesso andamento, dovuto al fatto che quasi tutti i siti all'interno del reticolo saranno colorati all'aumentare della probabilità di colorazione.

5.3.2 Osservabile P2

L'osservabile P2, definita come:

$$P2 = \frac{s_{max}}{pL^2}$$

dove:

- s_{max}, dimensione del cluster più grande.
- pL^2, numero di siti colorati all'interno del cluster.

Questa equazione rappresenta la probabilità di un sito colorato di appartenere al cluster più grande all'interno del reticolo. Viene riportato un grafico raffigurante l'andamento dell'osservabile P2 al cambiare della grandezza del reticolo e della probabilità di colorazione dei siti.

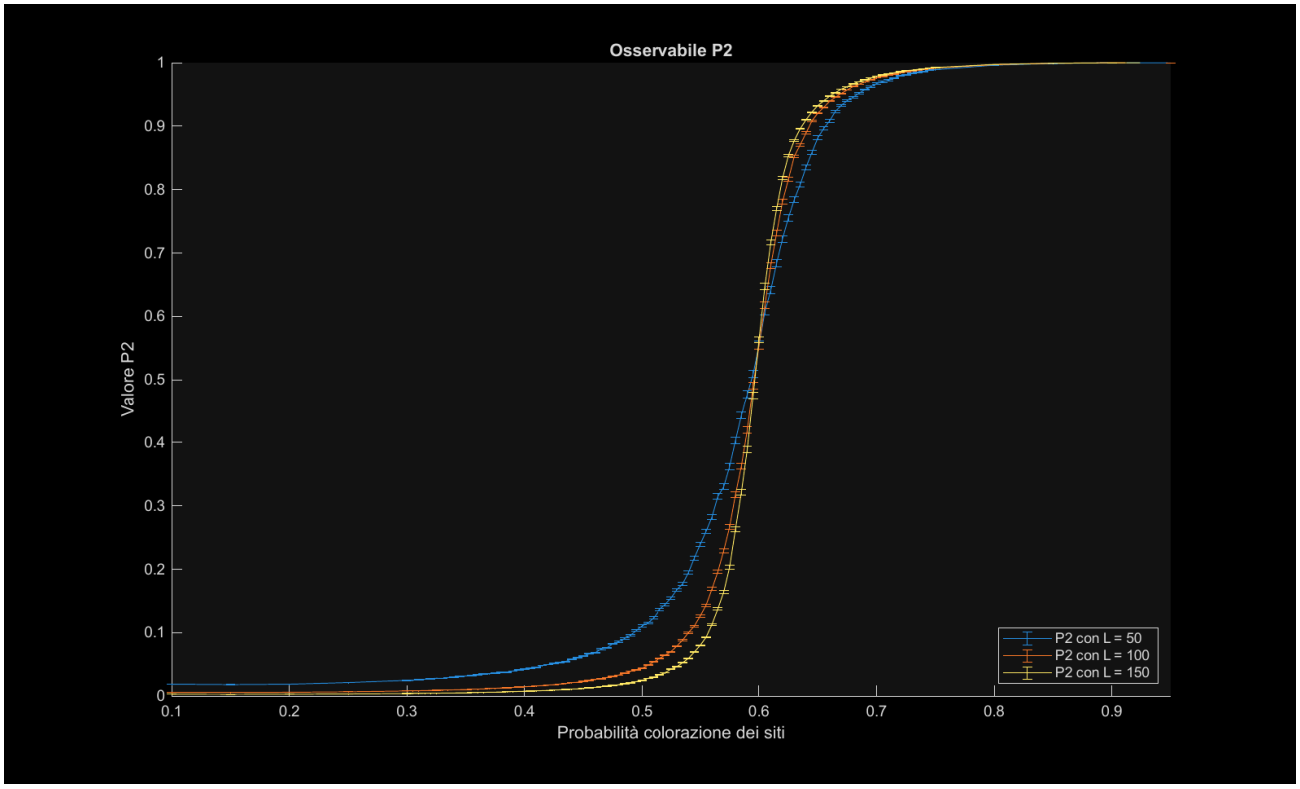


Figura 8: Osservabile P2

Come già è stato detto, l'osservabile P2 rappresenta la prboabilità di un sito colorato di appartenere al cluster più grande all'interno del reticolo. Possiamo notare che la curva inizia ad avere valori diversi da 0 con reticoli più piccoli, questo dovuto al fatto che reticoli più piccoli possono avere meno

cluster frammentati. Mentre per reticoli più grandi notiamo che la curva inizi a salire rapidamente tra 0.5 e 0.60. Anche in questo caso tutte le curve si incontrano ad un valore di probabilità di colorazione vicino a 0.6.

5.3.3 Osservabile P3

L'osservabile P3, definita come:

$$P3 = \frac{s_{max}}{\sum_s sn_s}$$

dove:

- s_{max}, dimensione del cluster più grande.
- $\sum_s sn_s$, somma pesata di ogni cluster.

Viene riportato qui sotto un grafico con gli andamenti dell'osservabile P3.

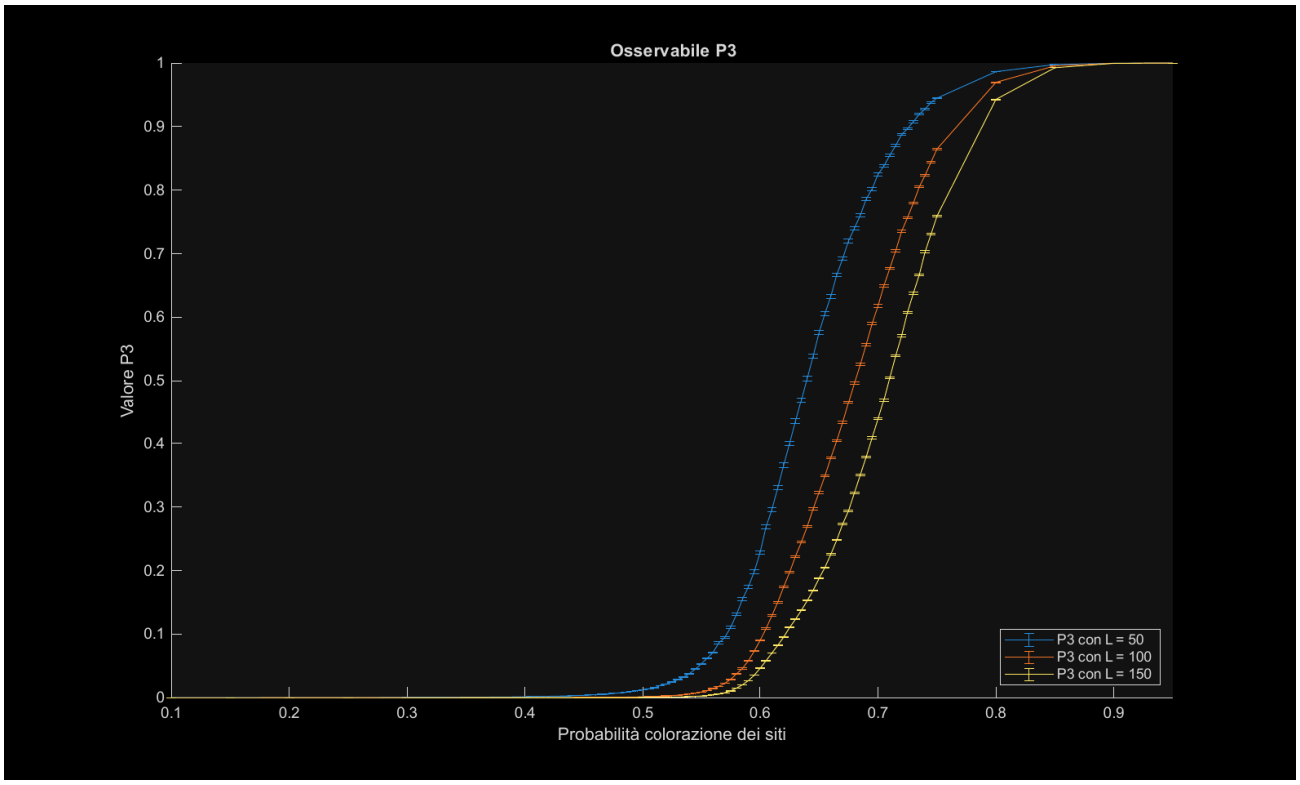


Figura 9: Osservabile P3

Possiamo notare dall'andamento del grafico che in questo caso le curve si incontrano ad una probabilità di colorazione dei siti pari a 0.8, quando iniziamo ad avere meno cluster sparsi per il reticolo e ci ritroviamo con un cluster unico su tutto il reticolo.

5.3.4 Osservabile RACS

L'osservabile RACS (Reduced Average Cluster Size), definita come:

$$RACS = \frac{\sum_{s \neq s_{max}} s(sn_s)}{\sum_{s'}(s'n_{s'})}$$

dove:

- $\sum_{s \neq s_{max}} s(sn_s)$, media pesata moltiplicata per la taglia del cluster; senza il cluster di dimensione massima
- $\sum_{s'}(s'n_{s'})$, media pesata di tutti i cluster.

Viene riportato di seguito un grafico con gli andamenti dell'osservabile RACS.

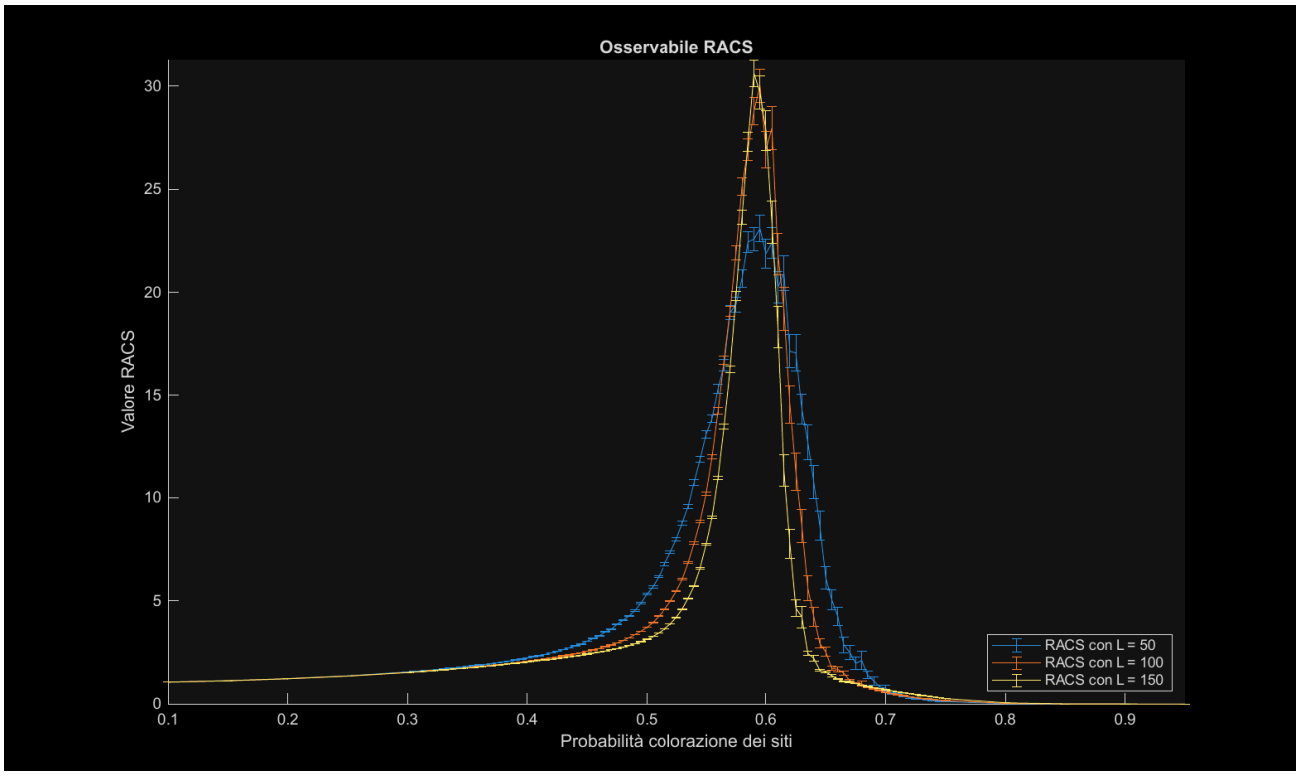


Figura 10: Osservabile RACS

Possiamo notare dal grafico che con l'aumento della probabilità di colorazione dei siti il valore del RACS tende ad un picco in corrispodenza della soglia di percolazione per poi iniziare a tendere ad un valore pari a 0 una volta superata la soglia. La tendenza a 0 è dovuta al fatto che ci saranno sempre meno cluster all'interno del reticolo finché non si forma un unico cluster.

6 Conclusioni

Con questo progetto è stato possibile studiare due diversi approcci per etichettare siti all'interno di un reticolo: L'algoritmo di cluster finding visto a lezione e l'algoritmo HK. Abbiamo visto come l'algoritmo visto a lezione risulti più efficiente su algoritmi di grandi dimensioni rispetto all'implementazione naive dell'algoritmo HK. Di coseguenza l'algoritmo HK è stato ottimizzato in modo da lavorare direttamente sui siti colorati risultando di gran lunga più veloce all'algoritmo visto a lezione. Quindi possiamo usare l'algoritmo visto durante le lezioni con reticoli di dimensioni contenute, mentre per simulazioni con reticoli di dimensioni elevate conviene utilizzare la versione ottimizzata dell'algoritmo HK.

È stata determinata la soglia di percolazione, le quattro osservabili P1, P2, P3 e RACS e studiati i loro andamenti al variare della dimensione del reticolo e del variare della probabilità di colorazione dei siti. Per la probabilità di percolazione abbiamo visto dal grafico che le curve tendono ad incontrarsi su valori molto vicini a 0.6 per quanto riguarda la probabilità di percolazione. Mentre per le quattro osservabili:

- P1 e P2 si incontrano circa ad un valore di probabilità di colorazione pari a 0.6, come per la probabilità di percolazione.
- P3 abbiamo visto che le curve tendono ad essere distanziate fino ad incontrarsi verso valori vicini ad 1 per la probabilità di colorazione dei siti.
- RACS invece tende ad avere un picco in corrispondenza di una probabilità di colorazione dei siti pari a circa 0.6. Oltre questo valore inizia a tendere verso zero, dovuto alla formazione di un unico cluster con l'aumentare della probabilità di colorazioen dei siti.

Pagina volutamente lasciata vuota

7 Appendice

Tabella 1: Confronto probabilità e errori, $L = 150$

Prob. Perc	Prob. perc TB	Prob. perc LR	Errore Prob. Perc	Errore TB	Errore LR
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Continua nella pagina successiva

Tabella 1 – continua dalla pagina precedente

Prob. Perc	Prob. perc TB	Prob. perc LR	Errore Prob. Perc	Errore TB	Errore LR
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0.001	0.001	0	0.00099	0.000999	0
0.003	0.002	0.001	0.00172	0.00141	0.00099
0.004	0.002	0.002	0.00199	0.00141	0.00141
0.006	0.004	0.002	0.00244	0.00199	0.00141
0.031	0.02	0.011	0.00548	0.00442	0.00329
0.058	0.038	0.026	0.00739	0.00604	0.00503
0.146	0.089	0.074	0.01116	0.00900	0.00827
0.251	0.148	0.155	0.01371	0.01122	0.01144
0.398	0.258	0.255	0.01547	0.01383	0.01378
0.591	0.387	0.423	0.01554	0.01540	0.01562
0.753	0.555	0.579	0.01363	0.01571	0.01561
0.875	0.731	0.718	0.01045	0.01402	0.01422
0.94	0.856	0.844	0.00750	0.01110	0.01147
0.981	0.922	0.923	0.00431	0.00848	0.00843

Continua nella pagina successiva

Tabella 1 – continua dalla pagina precedente

Prob. Perc	Prob. perc TB	Prob. perc LR	Errore Prob. Perc	Errore TB	Errore LR
0.996	0.963	0.972	0.00199	0.00596	0.00521
0.998	0.991	0.987	0.00141	0.00298	0.00358
1	0.995	0.997	0	0.00223	0.00172
1	0.998	0.999	0	0.00141	0.00099
1	1	0.999	0	0	0.00099
1	0.999	1	0	0.00099	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0